

# Dictionaries – creation, accessing values, adding and modifying key-value pairs, deleting items, and iteration.

A **dictionary** in Python is a collection of **key-value pairs**.

Keys must be unique and immutable. Values can be of any data type.

## 1. Creation of Dictionaries

Example :

```
# Using curly braces {}
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
# Using dict() constructor
marks = dict(math=90, eng=85, sci=95)
empty_dict = {} # empty dictionary
```

Output :

```
student → {'name': 'Alice', 'age': 20, 'course': 'Python'}
marks → {'math': 90, 'eng': 85, 'sci': 95}
empty_dict → {}
```

## 3. Adding and Modifying Key-Value Pairs

Example :

```
student = {'name': 'Alice', 'age': 20}
student['course'] = 'Python' # Adding new key-value pair
student['age'] = 21 # Modifying existing value
print(student)
```

Output :

```
{'name': 'Alice', 'age': 21, 'course': 'Python'}
```

## 5. Iteration over Dictionaries

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
# Iterating over keys
for key in student:
    print(key)
# Iterating over values
for value in student.values():
    print(value)
# Iterating over key-value pairs
for key, value in student.items():
    print(key, '→', value)
```

Output :

Keys:  
name  
age  
course

Values:  
Alice  
20  
Python

Items (key → value):  
name → Alice  
age → 20  
course → Python

## 2. Accessing Values

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
print(student['name']) # Access by key
print(student.get('age')) # Using get()
print(student.get('grade', 'Not found')) # Default value
```

Output :

```
Alice
20
Not found
```

## 4. Deleting Items

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
del student['age'] # Delete by key
removed = student.pop('course') # pop() removes and returns
print(student)
print(removed)
```

Output : {'name': 'Alice'}  
Python

### Key Points

- Dictionaries store data as key-value pairs.
- Keys must be unique and immutable (e.g., strings, numbers, tuples).
- Values can be of any data type.
- We can add, modify, and delete items using their keys.
- We can iterate over keys, values, or key-value pairs using loops.

Note : Accessing a key that doesn't exist using `[]` will raise a **KeyError**.  
Use `get()` method to avoid errors.

# Dictionaries - Creation

A dictionary in Python is a collection of **key-value pairs**.

Dictionaries are written using curly braces {}.

Each key must be **unique** and **immutable**.

## 1. Creating an Empty Dictionary

```
empty_dict = {}           # empty dictionary
print(empty_dict)         # Output: {}
```

## 2. Creating a Dictionary with Key-Value Pairs

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
print(student)           # Output: {'name': 'Alice', 'age': 20, 'course': 'Python'}
```

## 3. Creating a Dictionary Using dict() Constructor

```
marks = dict(math=90, eng=85, sci=95)
print(marks)             # Output: {'math': 90, 'eng': 85, 'sci': 95}
```

## 4. Creating a Dictionary from Sequence of Tuples

```
pairs = [('a', 1), ('b', 2), ('c', 3)]
d = dict(pairs)
print(d)                 # Output: {'a': 1, 'b': 2, 'c': 3}
```

### Key Points

- Dictionaries store data as key-value pairs.
- Keys must be unique and immutable.
- Values can be of any data type.

# Dictionaries – accessing values

We can access the values in a dictionary using keys.

There are two common ways to access values :

1. Using square bracket notation []
2. Using the get() method

## 1. Accessing Values Using Square Bracket Notation

We use the key inside square brackets [] to get the corresponding value.

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
name = student['name']           # Access by key
age = student['age']             # Access by key
print(name)
print(age)
```

Output :

Alice  
20

## 2. Accessing Values Using get() Method

The get() method returns the value for the given key.

If the key does not exist, it returns None (or a default value, if provided).

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
name = student.get('name')           # Returns 'Alice'
grade = student.get('grade')         # Returns None
city = student.get('city', 'Not found') # Returns 'Not found'
print(name)
print(grade)
print(city)
```

Output :

Alice  
None  
Not found

### Key Points

- Keys are used to access values in a dictionary.
- Use [] for direct access (raises KeyError if key is not found).
- Use get() for safe access (returns None or a default value if key is missing).

# Dictionaries – adding and modifying key-value pairs

We can add new key-value pairs to a dictionary or modify the values associated with existing keys.

## 1. Adding Key-Value Pairs

We can add a new key-value pair by assigning a value to a new key.  
If the key does not exist, it will be added to the dictionary.

Example :

```
student = {'name': 'Alice', 'age': 20}
student['course'] = 'Python'      # Adding new key-value pair
student['city'] = 'Patna'         # Adding another key-value pair
print(student)
```

Output :

```
{'name': 'Alice', 'age': 20,
'course': 'Python', 'city': 'Patna'}
```

## 2. Modifying Key-Value Pairs

We can modify the value of an existing key by assigning a new value to that key.

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}
student['age'] = 21                      # Modifying value of existing key
student['course'] = 'Data Science'       # Modifying another key
print(student)
```

Output :

```
{'name': 'Alice', 'age': 21,
'course': 'Data Science'}
```

## 3. Adding New Key and Modifying Existing Key Together

Example :

```
student = {'name': 'Alice', 'age': 20}
student['course'] = 'Python'             # Adding new key
student['age'] = 22                       # Modifying existing key
print(student)
```

Output :

```
{'name': 'Alice', 'age': 22,
'course': 'Python'}
```

### Key Points

- Add a new key-value pair using: `dict[key] = value` (if key does not exist).
- Modify the value of an existing key using: `dict[key] = new_value`.
- Keys must be unique and immutable.
- Values can be of any data type.

# Dictionaries – deleting items

We can delete items from a dictionary using different methods.

The most commonly used methods are :

1. del statement
2. pop() method
3. popitem() method
4. clear() method

## 1. Deleting an Item Using del Statement

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}  
del student['age']      # removes the key 'age' and its value  
print(student)
```

Output :

```
{'name': 'Alice', 'course': 'Python'}
```

## 2. Deleting an Item Using pop() Method

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}  
age = student.pop('age')  # removes 'age' and returns its value  
print(age)  
print(student)
```

Output :

```
20  
{'name': 'Alice', 'course': 'Python'}
```

## 3. Deleting a Random Item Using popitem() Method

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}  
item = student.popitem() # removes and returns last inserted item  
print(item)  
print(student)
```

Output :

```
('course', 'Python')  
{'name': 'Alice', 'age': 20}
```

## 4. Deleting All Items Using clear() Method

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}  
student.clear()      # removes all items from dictionary  
print(student)
```

Output :

```
{}
```

### Key Points

- del removes a specific key-value pair.
- pop(key) removes the item with the given key and returns its value.
- popitem() removes and returns the last inserted key-value pair.
- clear() removes all items from the dictionary.
- If the key does not exist, del and pop() will raise KeyError.

# Dictionaries - iteration.

We can iterate over dictionaries to access keys, values, or key-value pairs using loops.

## 1. Iterating Over Keys

By default, iterating over a dictionary gives us the keys.

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}  
for key in student:           # iterates over keys  
    print(key)
```

Output :

```
name  
age  
course
```

## 2. Iterating Over Values

We can iterate over all values in a dictionary using `.values()`.

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}  
for value in student.values(): # iterates over values  
    print(value)
```

Output :

```
Alice  
20  
Python
```

## 3. Iterating Over Key-Value Pairs (Items)

We can iterate over both keys and values using `.items()`.

Example :

```
student = {'name': 'Alice', 'age': 20, 'course': 'Python'}  
for key, value in student.items(): # iterates over key-value pairs  
    print(key, '→', value)
```

Output :

```
name → Alice  
age → 20  
course → Python
```

### Key Points

- Iterating over a dictionary allows us to access keys, values, or key-value pairs.
- `for key in dict` → iterates over keys.
- `for value in dict.values()` → iterates over values.
- `for key, value in dict.items()` → iterates over key-value pairs.
- Dictionaries are unordered, so the order of iteration is not guaranteed.